JTAGprobe_smp_pio_std_weact — ELF Anatomy and Runtime Walkthrough

What `build/JTAGprobe_smp_pio_std_weact.elf` contains, how it reaches `main`, and how one DAP packet becomes JTAG/SWD edges on the target

This document is a bottom-up explanation of the built firmware image. It starts at the ELF file, follows the reset path through the Pico SDK runtime and the µOS++ III SMP kernel, describes the two threads that matter (USB on core 0 and DAP on core 1), the lock-free rings between them, the CMSIS-DAP command decoder, and finally the two ways the probe drives pins: the PIO state machine for JTAG sequences and SIO bit-banging for SWD and the remaining JTAG operations.

Everything below refers to the ELF in `build/` as it exists at the time of writing:

Property	Value
File	`build/JTAGprobe_smp_pio_std_weact.elf`
SHA-256	`229d57d563c8625cff8c6e4cb028896beb2e3e12475bba5c09ff68772c38993a`
Format	ELF32, little-endian, ARM EABI5, statically linked, with debug info
Machine	ARM Cortex-M0+ (`-mcpu=cortex-m0plus -mthumb -mfloat-abi=soft`)
Entry point	`0x100001e9` (`_entry_point` , Thumb)
Build type	`copy_to_ram` (code executes from SRAM, loaded through flash)
Code size	52,524 B `.text` + 6,896 B `.data` (plus 256 B boot2, 396 B flash glue, 36 B binary info)
Static RAM	≈ 95.3 KiB (vector table + text + data + bss)
Heap	≈ 160.7 KiB (`__end__ .. __HeapLimit`)
Source baseline	commit `e17b7b0` plus the local WeAct changes in the working tree
Compiler flags	`-O3 -g -DNDEBUG -Wall -ffunction-sections -fdata-sections`

The figures are not estimates: they were read back from this exact ELF with `readelf` , `objdump` , `nm` and the linker map. The commands are listed in § 11 so the reader can reproduce every table.

Contents

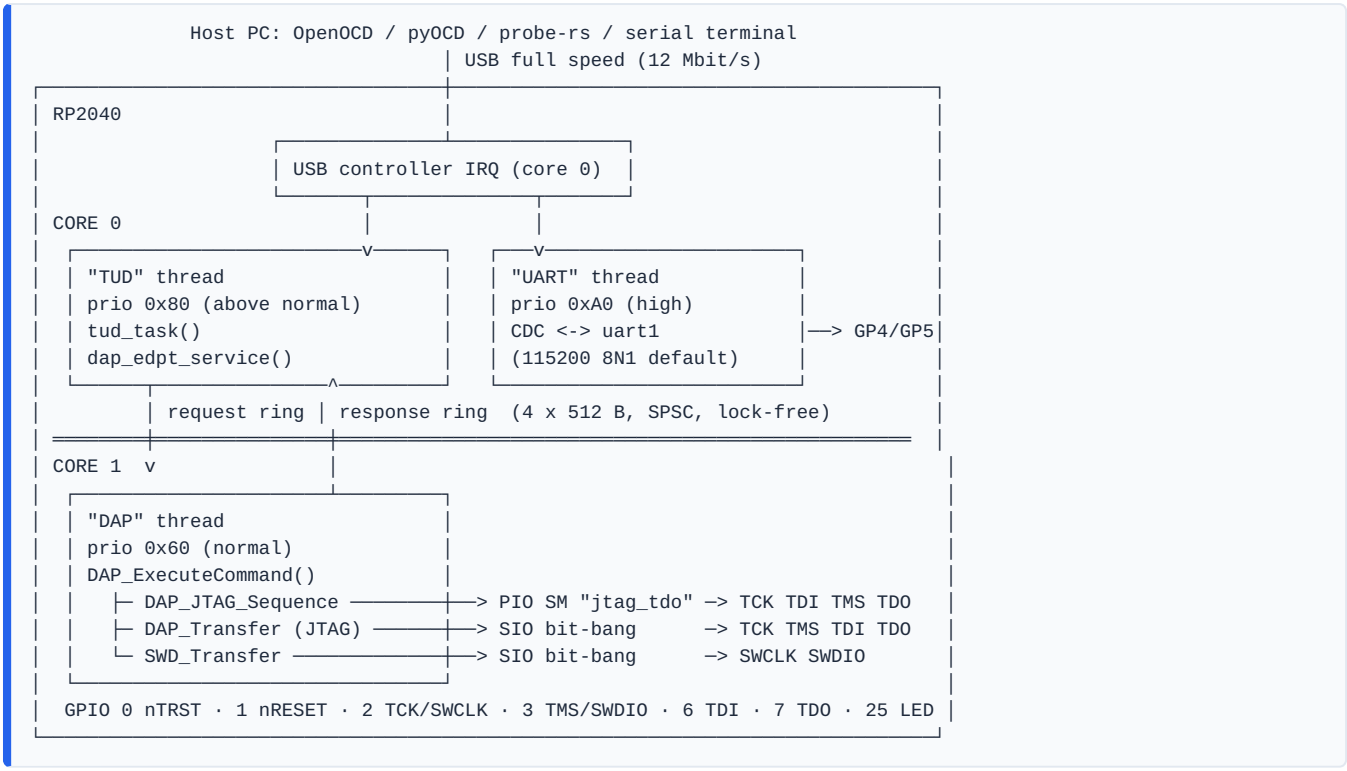
1. One-page mental model
2. ELF anatomy
3. Boot sequence: reset to `os_main`
4. Runtime model: threads, interrupts, synchronization
5. USB transport and the inter-core packet path
6. CMSIS-DAP command layer
7. Physical layer: pins, SIO bit-banging, PIO
8. End-to-end: one `DAP_JTAG_Sequence` request
9. Build configuration
10. Performance model and measurements
11. Inspecting and debugging the ELF
12. Source map and glossary

13. Appendix A: request-length grammar
14. Appendix B: constants and addresses

1. One-page mental model

The firmware is a **CMSIS-DAP v2 debug probe** with a **USB CDC-ACM UART bridge**, running on an RP2040 under **µOS++ III with one SMP scheduler across both Cortex-M0+ cores**. It is not a bare-metal superloop. The USB stack (tinyusb) is configured without an OS (`OPT_OS_NONE`) and is driven by a normal RTOS thread; the debug protocol runs in a separate thread on the other core.

Schema 1 — the whole machine in one picture



The five ideas that explain almost all of the code:

1. **Two cores, two jobs.** Core 0 owns USB and the UART bridge; core 1 does nothing but execute CMSIS-DAP commands. The target's semihosting traffic rides inside DAP packets, so bit-banging on core 1 overlaps the USB transfers of the previous and next packets on core 0.
2. **The packet path avoids the kernel.** Requests and responses cross the cores through two single-producer/single-consumer rings. The threads spin for up to 2 ms before blocking, so a busy transfer does not pay a context switch and a cross-core IPI per packet.
3. **tinyusb is serialized by one mutex.** `tud_task()` and the transfer/class callbacks run on core 0 with `usb_mutex` held. The DAP thread never calls `tud_task()`; it may only start an endpoint transfer itself if it wins a `try_lock` on the same mutex.
4. **JTAG sequences run in hardware.** `DAP_JTAG_Sequence` — the command OpenOCD uses for almost all scan work — is streamed into a PIO state machine that clocks TCK, drives TMS/TDI and samples TDO. The CPU only keeps the two FIFOs fed.
5. **Everything else bit-bangs.** `DAP_Transfer` (DP/AP access), IR scans, IDCODE reads, SWJ sequences and all of SWD go through single-cycle SIO writes with a calibrated delay loop.

2. ELF anatomy

2.1 ELF header

Field	Value	Meaning
`e_type`	`EXEC`	Fully linked, fixed addresses
`e_machine`	`ARM`	32-bit ARM
`e_entry`	`0x100001e9`	Pico SDK `_entry_point` (Thumb bit set)
`e_flags`	`0x5000200`	EABI5, soft-float ABI
Program headers	7 `PT_LOAD`	Flash boot region, RAM text, RAM data/bss, vector table, heap, two scratch stacks
Section headers	37	Includes debug sections (the file is not stripped)

The entry point is **not** `main`. It is the Pico SDK's second-stage entry stub in `.flashtext`, which:

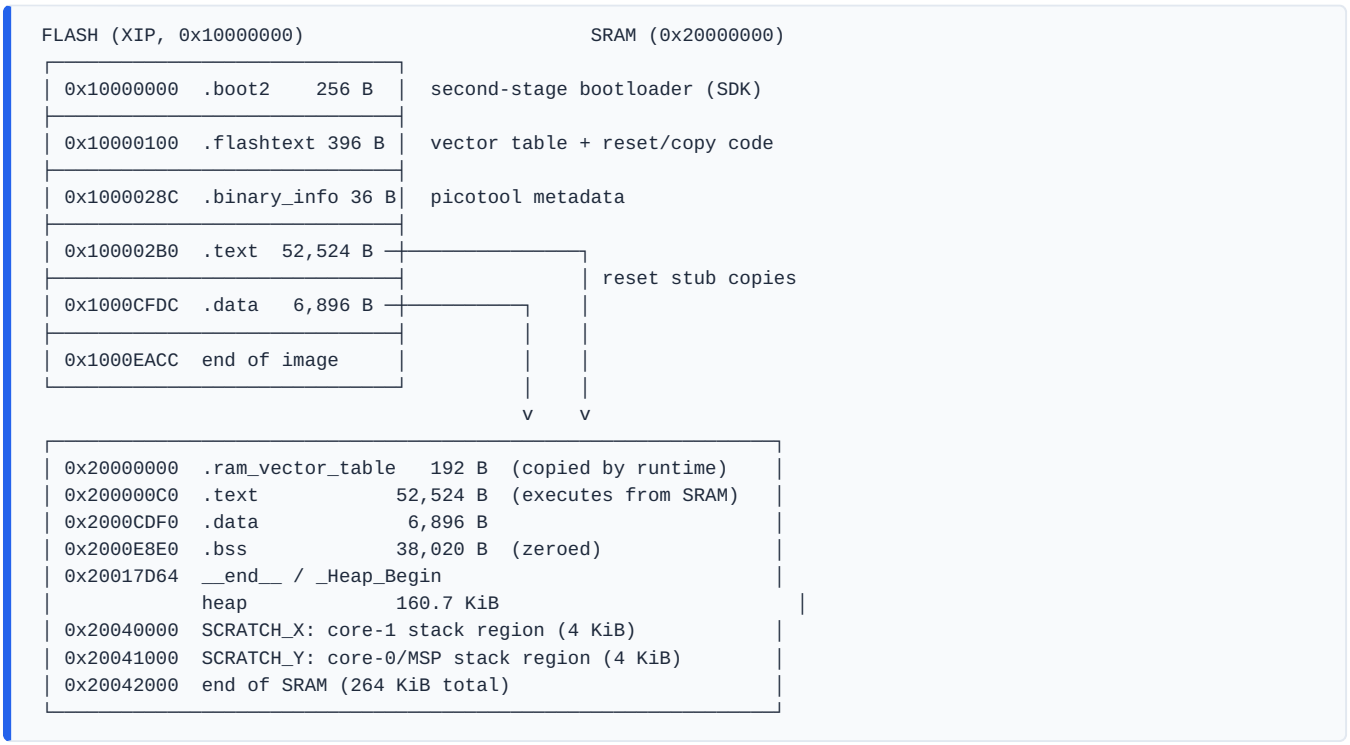
1. takes `r0 = 0` (the flash address of the vector table),
2. writes it to `VTOR` (`0xE000ED08`),
3. loads MSP and the reset vector from the table,
4. branches to the reset handler.

`main()` lives at `0x20006568` and is reached much later, after the SDK runtime has copied code into SRAM and run the C/C++ initializers.

2.2 Sections: two addresses per byte

Because the build uses `pico_set_binary_type(... copy_to_ram)`, most code has a **load address in flash** and a **virtual address in SRAM**. The reset stub copies each region before anything executes from it.

Schema 2 — load image (flash) and runtime image (SRAM)



Exact section table from `objdump -h`:

Section	VMA	LMA	Size	Role
`.boot2`	`0x10000000`	`0x10000000`	256	RP2040 second-stage boot (XIP/SSI setup)
`.flashtext`	`0x10000100`	`0x10000100`	396	Initial vector table, `_entry_point`, reset/copy code
`.binary_info`	`0x1000028C`	`0x1000028C`	36	picotool metadata
`.ram_vector_table`	`0x20000000`	—	192	Runtime vector table (48 entries)
`.text`	`0x200000C0`	`0x100002B0`	52,524	All code, runs from SRAM
`.data`	`0x2000CDF0`	`0x1000CFDC`	6,896	Initialized globals
`.bss`	`0x2000E8E0`	—	38,020	Zero-initialized globals
`.heap`	`0x20017D64`	—	2,048	Starter reservation; real limit is `__HeapLimit`

The `.data` segment is writable in RAM, which matters for one detail: `desc_configuration[]` is patched at runtime (the CDC `bmCapabilities` byte is changed to advertise `CAP_BREAK` support), so the descriptor table cannot live in flash.

2.3 Why `copy_to_ram`

The Pico SDK's `copy_to_ram` binary type deliberately places `.text` in SRAM. This firmware does no flash programming itself, so the motive is not self-update; it is **deterministic execution and timing**. SRAM has no XIP cache miss behaviour, and the bit-banging delay loops are calibrated against the CPU clock. A cache miss in the middle of a TCK half-period would stretch the clock. The cost is that the image is copied at every boot and consumes ~59 KiB of SRAM for code and initialized data.

2.4 The vector table

The initial table is in `.flashtext` at `0x10000100`; the SDK runtime copies it to `0x20000000` and points `VTOR` there (`runtime_init_install_ram_vector_table`). The entries that matter are the kernel's strong handlers, which replace the SDK's weak defaults because the SDK's exception names are renamed to the CMSIS names (`PICO_CMSIS_RENAME_EXCEPTIONS`).

Exception / IRQ	Vector address (RAM table)	Handler	Address in ELF
Reset	`0x20000004`	SDK `_reset_handler` (flash)	`0x100001F6`
NMI	`0x20000008`	SDK `NMI_Handler` (BKPT)	`0x100001CA`
HardFault	`0x2000000C`	SDK `HardFault_Handler` (BKPT)	`0x100001CC`
SVCall	`0x2000002C`	SDK `SVC_Handler` (BKPT)	`0x100001CE`
PendSV	`0x20000038`	**µOS++ context switch**	`0x20005031`
SysTick	`0x2000003C`	**µOS++ tick**	`0x20004FE5`
IRQ 15 `SIO_IRQ_PROC0`	`0x2000007C`	**µOS++ cross-core IPI (core 0)**	`0x20004F25`
IRQ 16 `SIO_IRQ_PROC1`	`0x20000080`	**µOS++ cross-core IPI (core 1)**	`0x20004F85`
other IRQs	—	SDK `unhandled_user_irq` / weak	`0x100001C1`

`PendSV` and `SysTick` are placed at the **lowest** NVIC priority (3 on a 2-bit-priority M0+), and the SIO FIFO IRQs likewise, so a context switch can never preempt a real device handler.

2.5 Key symbols and where they live

These addresses are useful when reading a map file or attaching a debugger.

Symbol	Address	Size	What it is
`DAP_ProcessCommand`	`0x200000C0`	0x17C4	DAP command decoder
`DAP_ExecuteCommand`	`0x20001884`	—	Batch (`ExecuteCommands`) wrapper
`JTAG_Transfer`	`0x20002244`	0x56E	JTAG DP/AP bit-bang transfer
`SWJ_Sequence`	`0x200027B4`	—	Generic SWJ bit-bang
`SWD_Transfer`	`0x20002B18`	0x67C	SWD packet/ACK/data bit-bang
`usb_thread`	`0x200031BC`	—	Core 0 USB loop
`os_main`	`0x20003218`	—	Probe initialization
`dap_edpt_xfer_cb`	`0x20003788`	—	tinyusb endpoint callback
`dap_edpt_service`	`0x20003930`	—	Re-arm OUT / start IN
`dap_thread`	`0x20003A44`	0x358	Core 1 DAP loop
`cdc_thread`	`0x20003F44`	—	Core 0 UART bridge loop
`pio_jtag_pins_to_sio`	`0x20004224`	—	Hand pins to SIO
`pio_jtag_init`	`0x2000433C`	—	Claim/configure PIO SM
`pio_jtag_sequences`	`0x200045A0`	—	Stream entries through PIO
`SysTick_Handler`	`0x20004FE4`	—	1 kHz tick
`PendSV_Handler`	`0x20005030`	—	Context switch
`main`	`0x20006568`	—	µOS++ weak `main()`
`tud_task_ext`	`0x2000A93C`	0x7EC	tinyusb device task

Thread stacks and shared objects:

Object	Address	Size	Notes
`dap_stack`	`0x2000EB58`	2048	DAP thread stack
`tud_stack`	`0x2000F628`	2048	USB thread stack
`uart_stack`	`0x2000FE28`	2048	UART thread stack
`idle1_stack`	`0x2000E950`	512	Core-1 idle thread
`DAPRequestBuffer`	`0x20010628`	512	DAP thread’s private request copy
`DAPResponseBuffer`	`0x20010828`	512	DAP thread’s private response copy
`DAP_Data`	`0x20010A28`	68	CMSIS-DAP state
`USBRequestBuffer`	`0x20010C70`	2064	Ring: data[4][512] + resp_len[4] + wptr + rptr

Object	Address	Size	Notes
`USBResponseBuffer`	`0x20011480`	2064	Same layout
`usb_mutex`	`0x20017CF4`	44	Serializes tinyusb
`usb_sem`	`0x20017D20`	24	Wakes USB thread
`dap_sem`	`0x20017578`	24	Wakes DAP thread
`uart_sem`	`0x20017C60`	24	Wakes UART thread
`dap_stats`	`0x2000F358`	48	12 packet-timing counters

The linker map also shows the big tinyusb allocations: `_vendord_itf` is `0x402C` bytes (16 KiB) and `_cdcd_itf` is `0x10A8` (4.2 KiB). Those, plus the two 2 KiB rings and four 2 KiB thread stacks, explain most of the 38 KiB `.bss`.

2.6 The reset copy loop, annotated

`objdump -d -j .flashtext` shows the whole mechanism. The interesting part is the `data_cpy_table`, a list of `(source, destination, end)` triples:

```
10000238  data_cpy_table:
10000238    .word 0x100002b0   ; .text source in flash
1000023c    .word 0x200000c0   ; .text destination in SRAM
10000240    .word 0x2000cdec   ; .text end
10000244    .word 0x1000cfdc   ; .data source
10000248    .word 0x2000cdf0   ; .data destination
1000024c    .word 0x2000e8e0   ; .data end
10000250    .word 0x1000eacc   ; bss "source" (never read)
10000254    .word 0x20040000   ; bss start
10000258    .word 0x20040000   ; bss end == start (empty range)
...
1000025c    .word 0x1000eacc   ; placeholder for stack/scratch areas
```

The reset handler walks this table, then zero-fills `.bss`, then calls `platform_entry`, which invokes `runtime_init()`, `main()`, and `exit()` (in that order; `exit()` is the Pico SDK's, retained instead of the kernel's).

3. Boot sequence: reset to `os_main`

Schema 3 — boot timeline

```
power-on / RUN reset
       │
       ▼
RP2040 boot ROM
  reads flash, validates the second-stage bootloader
       │
       ▼
.boot2  (0x10000000, 256 B)
  configures SSI/XIP for the flash chip, then jumps to 0x10000100
       │
       ▼
_entry_point (0x100001E8)
  VTOR = 0; MSP = [0x10000100]; PC = [0x10000104]
       │
       ▼
_reset_handler (0x100001F6)
  copy .text  flash 0x100002B0 -> SRAM 0x200000C0  (52,524 B)
  copy .data  flash 0x1000CFDC -> SRAM 0x2000CDF0  (6,896 B)
  zero .bss   0x2000E8E0 .. 0x20017D64            (38,020 B)
       │
       ▼
platform_entry
  runtime_init()
    ├─ runtime_run_initializers(): SDK clocks, GPIO, spinlocks,
    │    RAM vector table, alarm pool, per-core IRQ priorities, TLS
    └─ __init_array (sorted by priority)
         ├─ constructor 101: os_rp2040_initialize_free_store()
         │    gives the µOS++ kernel the heap __end__..__HeapLimit
         └─ other C++ constructors
  main()           µOS++ weak main, 0x20006568
       │
       ▼
µOS++ main()
  scheduler::initialize()
  create "main" thread (2 KiB, affinity core 0) -> _main_trampoline
  create core-0 idle thread (1 KiB)
  scheduler::start()
    MSP -> PSP, reset MSP to __StackTop, PendSV lowest priority,
    fake current thread, PendSV requested, IRQs enabled
       │
       ▼
PendSV -> first context switch -> "main" thread
       │
       ▼
_main_trampoline -> os_main(argc, argv)   0x20003218
```

3.1 The µOS++ `main()` and the first switch

`micro-os-plus/micro-os-plus-iii/src/rtos/os-main.cpp` provides a weak `main()`. The Pico SDK's `crt0` calls it normally. It:

1. prints the kernel banner through `trace::printf` (routed to the SDK stdio, which is UART0 by default on this build);
2. calls `scheduler::initialize()`;
3. creates the **`main` thread** with `OS_INTEGER_RTOS_MAIN_STACK_SIZE_BYTES` (2048) and, under `OS_USE_SMP_SCHEDULER`, pins it to core 0;
4. creates the core-0 **idle** thread (`OS_INTEGER_RTOS_IDLE_STACK_SIZE_BYTES`, 1024);
5. calls `scheduler::start()`.

The port's `scheduler::start()` does the architectural work (abridged; the real function also initialises the per-core lock state and the idle-thread array):

```
__disable_irq();
__set_PSP(__get_MSP());                       // PSP = current MSP
__set_CONTROL(__get_CONTROL() | CONTROL_SPSEL_Msk);
__ISB();
__set_MSP(*(uint32_t*)SCB->VTOR);             // reset MSP to __StackTop
NVIC_SetPriority(PendSV_IRQn, lowest_priority);
static os_thread_t fake_thread;               // first switch needs a "from"
...
scheduler::reschedule();                      // pend PendSV
__enable_irq();
for (;;) __NOP();
```

A fake “current” thread is installed so the first `switch_stacks()` has a node to unlink; it is never restored. When interrupts are enabled, PendSV runs on the MSP, saves the (fake) context, picks the highest-priority ready thread and returns to it. `main` is the only runnable thread at that moment, so it wins.

3.2 `os_main()`: probe initialization

`src/main.c` is deliberately short. The order matters (the `create_thread` calls are abridged; `...` stands for the stack size and attribute arguments):

```
int os_main(int argc, char *argv[]) {
    probe_info("Welcome to jtagprobe!\n");   // compiled out (probe_info = no-op)
    DAP_Setup();                             // CMSIS-DAP defaults, DAP_SETUP()
    board_init();                            // Pico SDK board init (stdio UART0)
    led_init();                              // GP25 + repeating timer
    usb_serial_init();                       // flash unique ID -> USB serial string

    os_mutex_create(&usb_mutex, "usb", NULL);
    os_semaphore_binary_create(&usb_sem, "usb", 0);
    os_semaphore_binary_create(&dap_sem, "dap", 0);
    os_semaphore_binary_create(&uart_sem, "uart", 0);

    tusb_init();                             // tinyusb device stack
    cdc_uart_init();                         // uart1 on GP4/GP5

    create_thread(&uart_thread_obj, "UART", cdc_thread, uart_stack, ...,
                  os_thread_priority_high, CORE0);
    create_thread(&tud_thread_obj, "TUD", usb_thread, tud_stack, ...,
                  os_thread_priority_above_normal, CORE0);
    create_thread(&dap_thread_obj, "DAP", dap_thread, dap_stack, ...,
                  os_thread_priority_normal, CORE1);

    os_rp2040_start_core1();                 // core 1 joins the scheduler
    os_thread_join(&tud_thread_obj, NULL);   // blocks forever
    return 0;
}
```

Two ordering details are not accidental:

- `DAP_Setup()` runs **before** `board_init()`. `DAP_SETUP()` explicitly drives nTRST (GP0) high, releasing the target TAP from reset. The comment in `CMakeLists.txt` explains that the Pico SDK’s UART0 TX on GP0 would otherwise hold it high implicitly; the explicit drive makes the behaviour independent of stdio configuration.
- `led_init()` starts a **repeating timer** rather than a thread. During a transfer the USB thread spins on core 0 at above-normal priority, and the DAP thread sleeps when idle; a normal LED thread could be starved in either state. The timer IRQ always runs.

3.3 Starting core 1

`os_rp2040_start_core1()` is the port’s bridge between the kernel SMP scheduler and the Pico SDK’s multicore launch:

1. It creates the **`idle1`** thread (512-byte stack, `secondary_idle_func`) and registers it in `scheduler::os_idle_thread_core[1]` *before* the launch, so core 1’s first reschedule always finds something to run.
2. It calls `multicore_reset_core1()` and `multicore_launch_core1_raw(port_smp_secondary_start, &__StackOneTop, scb_hw->vtor)`. Core 1 starts on the SDK’s `SCRATCH_X` stack with the same vector table.

3. `port_smp_secondary_start()` runs on core 1: it sets PendSV/SysTick to the lowest priority, starts SysTick at 1 kHz, drains the SIO FIFO (the launch handshake used it), enables `SIO_IRQ_PROC1`, installs a fake current thread, switches to PSP and reschedules. It then enters the scheduler; the DAP thread is ready and pinned to core 1, so it runs.
4. Back on core 0, `os_rp2040_start_core1()` drains the FIFO again and enables `SIO_IRQ_PROC0`. From now on each core has its own IPI channel.

The `main` thread then calls `os_thread_join(&tud_thread_obj, NULL)`. The USB thread never exits, so the main thread blocks forever and its 2 KiB stack is effectively idle for the rest of the run.

3.4 What “running” looks like

After boot there are six threads:

Core	Thread	State when host is idle
0	`main`	blocked in `os_thread_join`
0	`TUD`	sleeping on `usb_sem` (timed, 1 tick)
0	`UART`	sleeping on `uart_sem` (timed to the line-rate interval)
0	idle0	runs when nothing else is ready
1	`DAP`	sleeping on `dap_sem` (after a 2 ms spin)
1	idle1	runs when DAP sleeps

Nothing busy-waits indefinitely. The 2 ms spins exist only to cover the gap between packets during a transfer; after 2 ms of silence both sides block.

4. Runtime model: threads, interrupts, synchronization

4.1 Threads

Priorities are µOS++ numeric values (`OS_THREAD_PRIO_SHIFT == 4`); a larger number is more urgent.

Thread	Function	Core	Priority	Stack	Role
`UART`	`cdc_thread`	0	`0xA0` high	2048 @ `0x2000FE28`	USB CDC ↔ `uart1` bridge
`TUD`	`usb_thread`	0	`0x80` above normal	2048 @ `0x2000F628`	`tud_task()` + endpoint service
`DAP`	`dap_thread`	1	`0x60` normal	2048 @ `0x2000EB58`	`DAP_ExecuteCommand()`
`main`	`_main_trampoline`	0	`0x60` normal	2048 (heap)	startup, then join
`idle0`	kernel idle	0	`0x10` idle	1024 (heap)	core-0 idle
`idle1`	`secondary_idle_func`	1	`0x10` idle	512 @ `0x2000E950`	core-1 idle

The **UART thread deliberately preempts USB**. Its comment says characters get lost otherwise: the UART FIFO is only 32 bytes deep and the bridge polls at a rate derived from the line coding. On a single-core build the bridge had to preempt the bit-banging too; on this SMP build the DAP work is on the other core, so it cannot delay the bridge at all.

4.2 Synchronization objects

Object	Type	Created with	Purpose
`usb_mutex`	mutex	`os_mutex_create(..., NULL)`	Serializes every tinyusb call and the CDC request flags
`usb_sem`	binary semaphore, count 0	`os_semaphore_binary_create`	USB IRQ (`tud_event_hook_cb`) and DAP thread wake the USB thread
`dap_sem`	binary semaphore, count 0	idem	Endpoint callbacks wake the DAP thread; IN completion frees a response slot
`uart_sem`	binary semaphore, count 0	idem	CDC callbacks wake the UART thread; also used as the periodic timer

`os_mutex_create(..., NULL)` selects the µOS++ defaults: `protocol::inherit` (priority inheritance) with `mx_priority_ceiling = priority::highest` (the ceiling is only used by `protocol::protect`). Inheritance rarely comes into play here: the two blocking users of `usb_mutex` are on core 0, and the DAP thread only ever uses `try_lock`, which never blocks.

4.3 Interrupts

IRQ	Core	Priority	Handler	Work
SysTick	0	3 (lowest)	`os_systick_handler()`	advance RTOS clock, reschedule
SysTick	1	3	`reschedule()` only	pick up threads woken by core 0
PendSV	both	3	`PendSV_Handler`	context switch
`SIO_IRQ_PROC0`	0	3	`SIO_IRQ_PROC0_Handler`	drain FIFO, reschedule
`SIO_IRQ_PROC1`	1	3	`SIO_IRQ_PROC1_Handler`	drain FIFO, reschedule
USBCTRL_IRQ	0	SDK default	tinyusb `dcd_rp2040_irq`	endpoint events, calls `tud_event_hook_cb`
alarm/timer IRQ	0	SDK default	`alarm_pool_irq_handler`	LED blink, SDK timers

Only core 0 advances the system clock. Core 1's SysTick exists so that a thread woken by core 0 (via the scheduler's IPI) is picked up promptly; it does not call `os_systick_handler()`.

4.4 Scheduling and the spin-then-block policy

The kernel is a preemptive priority scheduler with per-thread CPU affinity. `OS_NCPU=2` and `OS_USE_SMP_SCHEDULER=1` are set by the RP2040 port's `CMakeLists.txt`. Each core has its own current-thread pointer, ready list selection and PendSV; a single recursive spinlock (`_smp_klock`, backed by SIO hardware spinlock `PICO_SPINLOCK_ID_OS1`) protects the kernel data structures.

The probe adds a latency optimization on top: **spin briefly, then block**.

Place	Spin threshold	Why
`dap_thread`, waiting for a request or a free response slot	`DAP_SPIN_US` = 2000 µs	During a transfer the next packet arrives well inside this; avoids two context switches + IPI per packet

Place	Spin threshold	Why
`usb_thread`, deciding whether to sleep	`USB_SPIN_US` = 2000 µs	Keeps polling `tud_task()` while DAP traffic is in flight
`uart_thread`	line-rate derived interval	Polls the UART FIFO often enough not to overrun it

`dap_wait_change()` implements the pattern:

```
static void dap_wait_change(volatile uint32_t *counter, uint32_t seen) {
    uint32_t start = time_us_32();
    while (*counter == seen) {
        if ((time_us_32() - start) >= DAP_SPIN_US) {
            os_semaphore_wait(&dap_sem);   // block; a post returns immediately
            return;
        }
    }
}
```

The semaphore is posted **before** the waiter sleeps in the race case, so the count is already 1 and `os_semaphore_wait` returns without blocking; the caller re-checks the counter. This is the standard “post cannot be lost” argument for a binary semaphore used as a wake-up flag.

4.5 The context switch

The RP2040 is ARMv6-M: no `BASEPRI`, no `STMDB` / `LDMIA` of `r8-r11`, no IT blocks. The port therefore uses the upstream Cortex-M0 approach: save `r4-r7` with `stmia`, move `r8-r11` and `EXC_RETURN` through `r3-r7`, and store them. PendSV runs on the MSP, so the outgoing thread’s PSP frame is quiescent as soon as `save_on_stack()` returns.

Schema 4 — thread stack frame

```
 lower addresses
 ┌──────────────────────────────────┐
 | r4  r5  r6  r7                   |  saved by save_on_stack()
 | r8  r9  r10 r11  EXC_RETURN      |  moved through r3-r7 on ARMv6-M
 ├──────────────────────────────────┤  <- saved SP (stack_ptr)
 | r0  r1  r2  r3                   |  hardware exception frame
 | r12 r14(LR) r15(PC) xPSR         |  (8 words, restored by exception return)
 └──────────────────────────────────┘
 higher addresses
```

`PendSV_Handler` is naked and contains exactly one call:

```
extern "C" void __attribute__((naked, used, optimize("1")))
PendSV_Handler(void) {
    os::rtos::port::scheduler::restore_from_stack(
        os::rtos::port::scheduler::switch_stacks(
            os::rtos::port::scheduler::save_on_stack()));
    asm volatile("bx lr");
}
```

`switch_stacks()` takes the kernel spinlock, publishes the outgoing `stack_ptr`, calls `internal_switch_threads()` (affinity-aware, per CPU), claims the incoming thread by nulling its `stack_ptr` so the other core cannot also pick it, releases the lock and returns the new SP. The lock is what makes one scheduler safe across two cores; it is held only for the selection, never across the thread’s execution.

4.6 Cross-core wake-up (IPI)

When a thread becomes ready on the other core, the scheduler calls `port_smp_ipi(cpu)`. The RP2040 port uses the SIO inter-core FIFO: if the TX FIFO is ready it writes the word `0x4D53` (`"SM"`) and executes `__SEV()`. The target core’s `SIO_IRQ_PROCn` handler drains the FIFO and pends its own PendSV. If the FIFO is full, the target already has a pending IPI, so the send is simply skipped — the channel is a wake-up, not a queue of work.

The packet path is designed so this IPI is **not** on the critical path when traffic is flowing. The consumer is on the other core, so a semaphore post does raise an IPI — but during a burst the consumer has usually not gone to sleep: it is still spinning, sees the index change, and proceeds without a block/wake pair. Only the first packet of a burst (or the first after a 2 ms gap) pays the cross-core wake-up.

4.7 Invariants worth remembering

1. **`tud_task()` only ever runs in the `TUD` thread on core 0.** The USB interrupt may queue events, but it does not process them.
2. **Once the threads are running, every tinyusb call made from thread context is under `usb_mutex`.** The one exception is the USB controller IRQ: it runs in interrupt context on core 0, queues events and posts `usb_sem`, but does not process them. (`tusb_init()` runs in `os_main` before any thread exists.) The mutex also covers the endpoint calls that the DAP thread makes through `dap_kick_usb()`; the `probe_os.h` comment that tinyusb is called "only from core 0" is a simplification — `tud_task()` is core-0-only, while `usbd_edpt_xfer()` may be issued by whichever thread wins the mutex.
3. **The DAP thread never calls `tud_task()`.** It only touches the rings and, optionally, `dap_edpt_service()`.
4. **Each ring index has exactly one writer.** `wptr` is produced, `rptr` is consumed. No lock is needed, only a `__dmb()` between a slot's data and the index that publishes it.
5. **`usb_mutex` is never held while blocking on a semaphore.** All code paths release it before sleeping.
6. **The UART thread owns `uart1`; CDC callbacks only leave requests.**

5. USB transport and the inter-core packet path

5.1 What the host sees

The device descriptor is:

Field	Value
VID:PID	`2E8A:000C` (Raspberry Pi / CMSIS-DAP Debug Probe)
`bcdUSB`	`0x0210` (USB 2.1, needed for the BOS descriptor)
`bcdDevice`	`0x0200` (firmware 2.0.0)
EP0 max packet	64
Manufacturer	`Raspberry Pi`
Product	`JTAG probe on Pico (CMSIS-DAP)`
Serial	16 hex digits from the RP2040 flash unique ID

Configuration (`desc_configuration[]`):

Interface	Class	Endpoints	Purpose
0	Vendor-specific, subclass 0, protocol 0	`0x04` OUT, `0x85` IN, bulk, 64 B	CMSIS-DAP v2
1	CDC communication	`0x81` IN, interrupt, 64 B	UART bridge control
2	CDC data	`0x02` OUT, `0x83` IN, bulk, 64 B	UART bridge data

The DAP interface is **WinUSB-compatible** through the Microsoft OS 2.0 descriptor set. Windows reads the BOS descriptor, then requests the MS OS 2.0 feature descriptor (vendor request 1, `wIndex == 7`), and installs the `DeviceInterfaceGUIDs` registry value `{CDB3B5AD-293B-4663-AA36-1AAE46463776}`. That is why the probe works on Windows without a driver package.

5.2 tinyusb configuration

Setting	Value	Consequence
`CFG_TUSB_OS`	`OPT_OS_NONE`	Effective build value; `src/tusb_config.h` defaults to `OPT_OS_PICO`, but CMake sets `TINYUSB_OPT_OS OPT_OS_NONE` before `pico_sdk_init()`. No tinyusb mutexes/OSAL; the probe supplies serialization
`CFG_TUSB_RHPORT0_MODE`	`OPT_MODE_DEVICE`	Device-only
`CFG_TUD_CDC`	1	CDC-ACM
`CFG_TUD_VENDOR`	1	CMSIS-DAP bulk
`CFG_TUD_CDC_RX_BUFSIZE`	128	USB → UART runway
`CFG_TUD_CDC_TX_BUFSIZE`	4096	UART → USB runway (Windows submits one URB)
`CFG_TUD_VENDOR_RX_BUFSIZE`	8192	DAP request buffering
`CFG_TUD_VENDOR_TX_BUFSIZE`	8192	DAP response buffering
`PICO_RP2040_USB_DEVICE_ENUMERATION_FIX`	1	SDK errata workaround
`TUD_OPT_RP2040_USB_DEVICE_UFRAME_FIX`	1	RP2040 USB frame fix
`PICO_USE_MALLOC_MUTEX`	0	Kernel's `malloc` already runs under the scheduler lock

The custom DAP class driver is registered through tinyusb's application hook:

```
usbd_class_driver_t const *usbd_app_driver_get_cb(uint8_t *driver_count) {
    *driver_count = 1;
    return &_dap_edpt_driver;
}
```

`dap_edpt_open()` validates the vendor interface, resets both ring indices, clears the endpoint state, opens the OUT and IN endpoints and arms the OUT endpoint with the first 64-byte buffer. The IN endpoint is not armed until the first response exists.

5.3 The two rings

`buffer_t` is a fixed-size SPSC ring:

```
typedef struct {
    uint8_t  data[DAP_PACKET_COUNT][DAP_PACKET_SIZE]; // 4 x 512
    uint16_t resp_len[DAP_PACKET_COUNT];              // meaningful on IN
    volatile uint32_t wptr;                           // producer only
    volatile uint32_t rptr;                           // consumer only
} buffer_t;
```

The indices are **free-running packet counters**, not masked values. Slot `n` is `data[n % 4]`; the ring is full when `wptr - rptr >= 4` and empty when they are equal. Unmasked counters avoid the classic "one slot wasted to distinguish full from empty" problem and make the occupancy count a simple subtraction. The 32-bit wrap is harmless because subtraction is modulo 2^{32}.

Schema 5 — request and response rings

```
producer (core 0, usb_mutex)                consumer (core 1, DAP thread)
─────────────────────────────                ─────────────────────────────
OUT callback:                                dap_thread():
  write slot[wptr % 4].data                    read slot[rptr % 4].data
  __dmb()                                      __dmb()
  wptr++                                       rptr++
  os_semaphore_post(&dap_sem)                  dap_kick_usb() if OUT stalled
       │                                            │
       ▼                                            ▼
USBRequestBuffer        data[0..3][0..511]   DAP_ExecuteCommand()
                        resp_len[0..3]
                        wptr, rptr
       ▲                                            ▲
       │                                            │
USBResponseBuffer       data[0..3][0..511]   response slot:
                        resp_len[0..3]         write data + resp_len
                        wptr, rptr             __dmb(); wptr++
                                                     │
IN callback:                                         │
  __dmb(); rptr++                                    │
  os_semaphore_post(&dap_sem)  <─────────────────────┘
```

Each direction has its own ring because requests and responses are produced and consumed independently. `USBRequestBuffer` is written by the OUT endpoint callback and read by the DAP thread; `USBResponseBuffer` is written by the DAP thread and read by the IN side. `resp_len` is only used on the response ring; it is stored in the same cache-friendly structure and indexed with the same slot number.

The `__dmb()` calls are the entire synchronization protocol. On Cortex-M0+ a `DMB` orders the slot writes before the index store, so the consumer that sees the new index also sees the data. No atomic read-modify-write is needed because only one side writes each index.

5.4 Receiving a request: the OUT state machine

A DAP packet is **512 bytes**, but the full-speed bulk endpoint moves **64 bytes** per USB packet. The host does not send a zero-length packet after a request that is an exact multiple of 64; a USB transfer ends on a short packet. The firmware therefore cannot simply arm a 512-byte buffer and wait. It arms the OUT endpoint **one 64-byte packet at a time** and parses the DAP command lengths to know where the request ends.

Schema 6 — OUT request assembly

```
dap_edpt_open():
  out_len = 0
  arm OUT -> &slot[wptr%4][0], 64 bytes
       │
       ▼
USB packet arrives (64 B) -> dap_edpt_xfer_cb(OUT)
       │
       ├─ out_len += xferred_bytes
       │
       ├─ xferred == 64 && out_len < 512 && dap_req_len(slot, out_len) > out_len
       │      └─ arm OUT -> slot + out_len, 64 bytes        (need more)
       │
       ├─ out_len == 0
       │      └─ stray ZLP: re-arm, return
       │
       └─ request complete:
              out_len = 0
              record timing
              __dmb()
              USBRequestBuffer.wptr++
              if (!buffer_full) arm OUT for the next request
              else             out_stalled = true
              os_semaphore_post(&dap_sem)
```

`dap_req_len()` is a miniature parser for the CMSIS-DAP command encoding. It returns the total length of the command starting at `p`, the sentinel `0xFFFFFFFF` if more bytes are needed, or `1` for an unknown command (the fixed IDs are all that OpenOCD sends). It handles:

- fixed lengths: Disconnect/TransferAbort/ResetTarget/SWO_Status = 1, Info/Connect/SWD_Configure/JTAG_IDCODE/… = 2, HostStatus/Delay/SWO_Data = 3, SWJ_Clock/SWO_Baudrate = 5, TransferConfigure/WriteABORT = 6, SWJ_Pins = 7;
- variable counts: `Transfer` (per-transfer request/data bytes), `TransferBlock` (16-bit count plus optional 4-byte data per item), `SWJ_Sequence` / `SWD_Sequence` / `JTAG_Sequence`;
- recursive `QueueCommands` / `ExecuteCommands`, each containing sub-commands whose lengths are parsed the same way.

If the ring is full when a request completes, the endpoint is left **unarmed** (`out_stalled = true`) rather than overwriting a slot. `dap_edpt_service()` re-arms it as soon as the DAP thread frees a slot. This back-pressure is visible to the host as NAKs, which is exactly the desired USB flow control.

5.5 Sending a response: the IN state machine

A response is at most 512 bytes. The IN endpoint sends the whole response in one transfer, which tinyusb splits into 64-byte USB packets. Two details are handled explicitly:

1. **Zero-length packet (ZLP):** if the response length is a multiple of 64 and shorter than 512, the host would otherwise wait for more data. The driver sends a ZLP after the data transfer (`in_zlp = true`, then a `usbd_edpt_xfer(..., NULL, 0)` on completion).
2. **One transfer at a time:** `in_busy` guards the endpoint. On completion the driver frees the slot (`rptr++`), posts `dap_sem` (in case the DAP thread is waiting for a free response slot) and immediately starts the next response if one is queued.

```
static void dap_edpt_start_in(void) {
    if (!in_busy && !buffer_empty(&USBResponseBuffer)) {
        __dmb();
        in_busy = true;
        uint16_t len = USBResponseBuffer.resp_len[RD_IDX(USBResponseBuffer)];
        in_zlp = (len % DAP_EP_SIZE) == 0 && len < DAP_PACKET_SIZE;
        usbd_edpt_xfer(_rhport, _in_ep_addr, RD_SLOT_PTR(USBResponseBuffer), len);
    }
}
```

5.6 Who starts the IN transfer?

Normally the `TUD` thread does, right after `tud_task()`:

```
os_mutex_lock(&usb_mutex);
tud_task();
dap_edpt_service();          // re-arm OUT if stalled; start IN if response ready
bool more = tud_task_event_ready() || dap_edpt_active();
os_mutex_unlock(&usb_mutex);
if (!more) os_semaphore_timed_wait(&usb_sem, 1);
```

But the DAP thread can also do it itself, provided it can take the mutex without blocking:

```
static void dap_kick_usb(void) {
    if (os_mutex_try_lock(&usb_mutex) == os_ok) {
        dap_edpt_service();
        os_mutex_unlock(&usb_mutex);
    } else {
        os_semaphore_post(&usb_sem);   // let the TUD thread do it
    }
}
```

This is the difference between the first SMP build (which woke the USB thread for every packet and was ~4× slower) and the current one: in the common case the DAP thread publishes the response and immediately starts the IN transfer, without a

sleep/wake pair. If the USB thread happens to hold the mutex (e.g. inside `tud_task()`), the DAP thread posts `usb_sem` and lets it finish.

5.7 The CDC-ACM UART bridge

The CDC callbacks also run in the `TUD` thread, inside `tud_task()`. They do **not** touch the UART. They record a request and post `uart_sem`:

Callback	Request recorded	Applied by
`tud_cdc_line_coding_cb`	`pending_coding`, `coding_pending`	`cdc_apply_line_coding()`
`tud_cdc_line_state_cb`	`uart_active = dtr OR rts`	polling decision
`tud_cdc_send_break_cb`	`pending_break`, `break_pending`	`cdc_apply_break()`

The UART thread's period is derived from the line rate so that it polls the 32-byte hardware FIFO often enough:

```
uint32_t micros = (1000 * 1000 * 16 * 10) / MAX(bit_rate, 1);
interval = MAX(1, micros / ((1000 * 1000) / TICK_RATE_HZ));
```

At 115200 baud that is roughly one tick (1 ms), i.e. it wakes about every millisecond, drains the FIFO into USB and writes up to 16 bytes back. The `DEBOUNCE_MS` logic suppresses spurious wake-ups after a line-coding change.

`cdc_apply_line_coding()` deinitializes and reinitializes `uart1`, which is why it runs on the UART thread and not in the USB callback. Timed breaks store an expiry timestamp (`os_sysclock_now() + wValue ms`) and are cleared either by the thread or on disconnect.

6. CMSIS-DAP command layer

6.1 Packet and response shape

A DAP request is a command byte followed by command-specific bytes. A response begins with the same command byte. Most commands then return a status byte (`DAP_OK == 0` or `DAP_ERROR == 0xFF`) and their data.

```
request:   [ cmd ][ arg0 ][ arg1 ] ...
response:  [ cmd ][ status ][ data ] ...
```

`DAP_ProcessCommand()` returns a packed value:

```
bits 31..16 : number of request bytes consumed
bits 15..0  : number of response bytes produced
```

That packing is what makes `DAP_ExecuteCommands` and the length parser possible: the caller can walk a stream of commands without knowing each command's layout in advance.

6.2 Dispatch

`DAP_ProcessCommand()` handles the vendor range first (`0x80..0x9F`) and then switches on the command byte. The probe implements:

ID	Command	Notes
`0x00`	`DAP_Info`	Vendor/product/serial/firmware/capabilities/ packet size/count/timestamp
`0x01`	`DAP_HostStatus`	LEDs; both LEDs are optional in this board config
`0x02`	`DAP_Connect`	SWD or JTAG; `PORT_*_SETUP()`

ID	Command	Notes
`0x03`	`DAP_Disconnect`	all pins high-Z
`0x04`	`DAP_TransferConfigure`	idle cycles, retry count, match retry
`0x05`	`DAP_Transfer`	DP/AP transfers; core of debugging
`0x06`	`DAP_TransferBlock`	repeated transfers with a 16-bit count
`0x07`	`DAP_TransferAbort`	**not handled**: there is no `case` in `DAP_ProcessCommand`, so it falls through to `ID_DAP_Invalid`; the `DAP_TransferAbort` flag is never set
`0x08`	`DAP_WriteABORT`	JTAG ABORT register or SWD DP ABORT
`0x09`	`DAP_Delay`	busy delay in microseconds
`0x0A`	`DAP_ResetTarget`	`RESET_TARGET()` returns 0 (no custom sequence)
`0x10`	`DAP_SWJ_Pins`	direct pin drive/read with optional wait
`0x11`	`DAP_SWJ_Clock`	sets bit-bang delay and PIO divider
`0x12`	`DAP_SWJ_Sequence`	generic bit sequence, always SIO
`0x13`	`DAP_SWD_Configure`	turnaround and data phase
`0x1D`	`DAP_SWD_Sequence`	SWD sequences, always SIO
`0x14`	`DAP_JTAG_Sequence`	**PIO fast path** when enabled
`0x15`	`DAP_JTAG_Configure`	IR lengths before/after each TAP
`0x16`	`DAP_JTAG_IDCODE`	IR scan + IDCODE read (SIO)
`0x7E`	`DAP_QueueCommands`	converted in place to `ExecuteCommands`
`0x7F`	`DAP_ExecuteCommands`	batch of sub-commands
`0x80`	vendor 0	packet-timing counters (probe-specific)

SWO commands (`0x17..0x1C` and `0x1E`; `0x1D` is `DAP_SWD_Sequence`, not SWO) are compiled out because `SWO_UART == 0` and `SWO_MANCHESTER == 0`; `src/SWO.c` is still in the build but produces no command handlers under these settings.

One quirk worth calling out: `ID_DAP_TransferAbort` is defined, and every transfer retry loop tests `DAP_TransferAbort`, but no command handler sets it and the command is absent from the dispatch switch. A host that sends `0x07` gets `ID_DAP_Invalid`; aborting a long retry loop therefore relies on the retry count, not on this command.

6.3 Batching: QueueCommands / ExecuteCommands

A host can queue several commands and ask the probe to execute them atomically from the target's point of view. The DAP thread implements this without a separate queue:

```
n = USBRequestBuffer.rptr;
while (USBRequestBuffer.data[n % DAP_PACKET_COUNT][0] == ID_DAP_QueueCommands) {
    USBRequestBuffer.data[n % DAP_PACKET_COUNT][0] = ID_DAP_ExecuteCommands;
    n++;
    while (n == USBRequestBuffer.wptr)          // wait for the next packet
        dap_wait_change(&USBRequestBuffer.wptr, n);
}
```

Queued commands sit in consecutive ring slots. The first non-Queue command terminates the batch; by then every queued slot has been rewritten to `ExecuteCommands`, and `DAP_ExecuteCommand()` walks them in order. This is why a DAP

"packet" is not necessarily one command: a 512-byte request can carry a whole batch, and the batch can span multiple USB packets.

6.4 `DAP_Info` values

Info ID	Returned value
Vendor / Product / Serial	All return 0 bytes; the USB serial string is built separately in `get_serial.c`, not through `DAP_Info`
Firmware version	`"2.0.0"`
Capabilities	`0x33`: SWD (bit 0), JTAG (bit 1), atomic commands (bit 4), timestamp (bit 5)
Packet size	`512`
Packet count	`4`
Timestamp clock	`1,000,000` Hz (`time_us_32`)
SWO buffer size	Not returned (SWO disabled)

6.5 `DAP_Transfer`: the posted-read algorithm

`DAP_Transfer` is the command that actually reads and writes the target's Debug Port and Access Ports. The implementation is the standard CMSIS-DAP algorithm, and the details matter for understanding the bit-banging:

- Each transfer item is a request byte: `APnDP` (bit 0), `RnW` (bit 1), `A2` (bit 2), `A3` (bit 3), plus optional `MATCH_VALUE` (bit 4), `MATCH_MASK` (bit 5) and `TIMESTAMP` (bit 7).
- For JTAG, the request selects IR `JTAG_DPACC` (`0x0A`) or `JTAG_APACC` (`0x0B`) and the IR is rescanned only when it changes.
- AP reads are **posted**: the probe issues the read, then reads the previous result from `DP_RDBUFF` on the next transaction. The last read is flushed by an extra `DP_RDBUFF` read. This is why a single AP read produces its data one transaction later and why `post_read` appears throughout the code.
- WAIT responses are retried up to `DAP_Data.transfer.retry_count` (default 100). FAULT and protocol errors end the command with the corresponding ACK bits in the response.
- `MATCH_VALUE` reads repeatedly (up to `match_retry`) until `(data & match_mask) == match_value`, then reports `DAP_TRANSFER_MISMATCH` if it never matched. A write item with `MATCH_MASK` stores its 32-bit value in `DAP_Data.transfer.match_mask` instead of writing the target.
- `idle_cycles` clocks extra TCK/SWCLK cycles with TMS/SWDIO low after a successful transfer.
- `TIMESTAMP` stores `TIMESTAMP_GET()` (microseconds) in the response.

The JTAG variant is `DAP_JTAG_Transfer()` (in `DAP.c`, inlined into `DAP_ProcessCommand` at `-O3`); the SWD variant is `DAP_SWD_Transfer()`. Both call the corresponding `JTAG_Transfer()` / `SWD_Transfer()` per item.

6.6 `DAP_TransferBlock`

`DAP_TransferBlock` is the bulk form: a 16-bit count and one request byte, then either `count` 32-bit values to write or `count` 32-bit results. It is what makes `load_image` fast:

- **Read:** post one read, then loop reading the previous value; the last iteration reads `DP_RDBUFF` to flush the pipeline.
- **Write:** loop writing the data; after the last write, read `DP_RDBUFF` to check that the final write completed without a WAIT/FAULT.

The response starts with a 16-bit count and a status byte.

6.7 `DAP_SWJ_Clock`

This command sets two independent things:

```
if (clock >= MAX_SWJ_CLOCK(DELAY_FAST_CYCLES)) {
    DAP_Data.fast_clock = 1; DAP_Data.clock_delay = 1;
} else {
    DAP_Data.fast_clock = 0;
    delay = ((CPU_CLOCK/2) + (clock-1)) / clock;
    if (delay > IO_PORT_WRITE_CYCLES) {
        delay -= IO_PORT_WRITE_CYCLES;
        delay = (delay + (DELAY_SLOW_CYCLES-1)) / DELAY_SLOW_CYCLES;
    } else delay = 1;
    DAP_Data.clock_delay = delay;
}
pio_jtag_set_clock(clock);
```

- `DAP_Data.clock_delay` controls the **SIO bit-bang** loops (`PIN_DELAY_SLOW`). Above the fast threshold the loops are replaced by `PIN_DELAY_FAST()` (0–3 NOPs).
- `pio_jtag_set_clock()` sets the **PIO divider**: `div = clk_sys / (4 * clock)`, clamped to `[1, 65535]`. With a 125 MHz system clock, `div = 1` at 31.25 MHz, so the PIO can produce at most 31.25 MHz TCK regardless of the requested speed.

6.8 The vendor timing command (`0x80`)

`DAP_ProcessVendorCommand()` implements one probe-specific command. A request is `[0x80, sel]`; the response is `[0x80, u32 value]` (5 bytes total: the command byte plus four little-endian data bytes). The fields are:

`sel`	Field	Meaning
0	`packets`	Requests executed since reset
1	`queue_us`	OUT received → DAP thread starts the request
2	`exec_us`	Total `DAP_ExecuteCommand()` time
3	`exec_max_us`	Worst single execution
4	`slot_wait_us`	DAP thread waiting for a free response slot
5	`inq_us`	Response ready → IN transfer started
6	`in_us`	IN transfer started → complete
7	`in_n`	IN transfers completed
8	`gap_us`	Sum of OUT→OUT gaps shorter than 10 ms
9	`gap_n`	Number of those gaps
10	`gap_idle_n`	Number of OUT gaps ≥ 10 ms (host idle time)
11	`out_stalls`	Times the OUT endpoint was left unarmed because the ring was full
`0xFF`	—	Zero all counters, return 0

From OpenOCD this is read with `cmsis-dap cmd 0x80 0x00` (or `0x80 0x01` to read and reset). It is the measurement path used in § 10.

7. Physical layer: pins, SIO bit-banging, PIO

7.1 Pin map

All signals are on the left header, pins 1–10, matching a 10-way JTAG ribbon:

Header	GPIO	Signal	Direction	Driven by
1	0	nTRST	Output, held high	SIO (fixed high at `DAP_SETUP`)
2	1	nRESET	Open-drain	SIO
3	—	GND	—	—
4	2	SWCLK / TCK	Output	PIO side-set 0, or SIO
5	3	SWDIO / TMS	Bidirectional	PIO set 0 (JTAG), SIO (SWD)
6	4	UART TX	Output	`uart1`
7	5	UART RX	Input	`uart1`
8	—	GND	—	—
9	6	TDI	Output	PIO out 0, or SIO
10	7	TDO	Input (pull-up in PIO JTAG mode)	PIO in 0 with input sync bypass
—	25	Status LED	Output	SIO + repeating timer

`PROBE_UART_INTERFACE` is `uart1`, not the SDK's default `uart0`: GP4/GP5 are UART1's pins. UART0 remains available for `printf` / `trace` on GP0/GP1, which is why `board_init()`'s stdio configuration matters for nTRST.

7.2 Pin setup by mode

Mode	Action
`DAP_SETUP()`	GPIO init all six; TCK/TMS/TDI outputs high; TDO input; nRESET input with pull-up and latch high; nTRST output high
`PORT_JTAG_SETUP()`	`pio_jtag_init(g_pio_jtag.clock_hz)` — PIO owns TCK/TDI/TMS; TDO input; nTRST high; nRESET input/pull-up
`PORT_SWD_SETUP()`	`pio_jtag_deinit()`; SWCLK and SWDIO outputs high; nRESET output high; TDI and nTRST high-Z
`PORT_OFF()`	`pio_jtag_deinit()`; all six pins high-Z

`pio_jtag_deinit()` disables the SM, removes the program, unclaims it and returns the pins to SIO high-Z. This is important for SWD: TDI and nTRST must not fight the target.

One board-specific caveat: in this configuration `PIN_nTRST_OUT()` is a no-op and `PIN_nTRST_IN()` always returns 0 (both are marked "Not available" in `DAP_config.h`). nTRST is driven high once by `DAP_SETUP()` and then left there; `DAP_SWJ_Pins` cannot toggle it, and its bit in the `DAP_SWJ_Pins` response is always 0. nRESET, by contrast, is fully controlled (open-drain: drive low to assert, switch to input with pull-up to release).

7.3 SIO bit-banging

All non-PIO paths use the RP2040's single-cycle GPIO aliases:

```
__STATIC_FORCEINLINE void PIN_SWCLK_TCK_SET(void) { gpio_set_mask(MASK_SWCLK_TCK); }
__STATIC_FORCEINLINE void PIN_SWCLK_TCK_CLR(void) { gpio_clr_mask(MASK_SWCLK_TCK); }
__STATIC_FORCEINLINE uint32_t PIN_TDO_IN(void)   { return (gpio_get_all() >> 7) & 1;
```

The JTAG bit-bang macro pair is:

```
#define JTAG_CYCLE_TDIO(tdi,tdo) \
  PIN_TDI_OUT(tdi);              \
  PIN_TCK_CLR();                 \
  PIN_DELAY();                   \
  tdo = PIN_TDO_IN();            \
  PIN_TCK_SET();                 \
  PIN_DELAY()
```

so TDI changes while TCK is low, TDO is sampled just before the rising edge, and the delay loop (`PIN_DELAY_SLOW(DAP_Data.clock_delay)`) sets the half period. The SWD path is the same idea with a turnaround phase and an ACK window:

- write 8 request bits (start, APnDP, RnW, A2, A3, parity, stop, park);
- release SWDIO (`PIN_SWDIO_OUT_DISABLE()`), clock the configured turnaround;
- read 3 ACK bits;
- on OK, read or write 32 data bits plus parity, then turnaround back;
- on WAIT/FAULT, optionally clock the data phase, then return the ACK;
- on a protocol error, clock a back-off sequence and return the raw ACK.

The SWD code is `src/SW_DP.c`, generated twice by macro: `SWD_TransferFast` (no delay) and `SWD_TransferSlow` (delay loop). `DAP_Data.fast_clock` selects.

7.4 The PIO engine

`src/jtag.pio` defines a single program, `jtag_tdo`. `pio_jtag_init()`:

1. claims an unused state machine on `pio0`, falling back to `pio1`;
2. adds the program and gets its instruction offset;
3. configures:
   - `out` pins → TDI (1 pin),
   - `set` pins → TMS (1 pin),
   - `in` pins → TDO (1 pin),
   - `sideset` pins → TCK (1 pin);
4. sets shift directions:
   - OUT: shift right, **no autopull**, threshold 8,
   - IN: shift right, **autopush**, threshold 8;
5. computes the divider `div = clk_sys / (4 * clock_hz)`, clamped;
6. drives TCK/TDI/TMS high and sets their directions;
7. routes those three pins to the PIO (`pio_gpio_init`), configures TDO as a plain SIO input with pull-up, and sets the PIO input sync bypass for TDO;
8. drives nTRST high and leaves nRESET as an open-drain input with pull-up;
9. initializes and enables the SM.

The **no autopull** choice is subtle: with autopull, the PIO would treat the next entry's header word as TDI data as soon as the OSR emptied. Instead the program pulls explicitly with `pull ifempty`, and the CPU controls exactly when a data byte is consumed. The **autopush** choice lets the PIO push a TDO word every 8 bits without CPU intervention.

7.5 The PIO program, instruction by instruction

```
.wrap_target
    pull block          side 1   ; fetch header; TCK high
    out x, 7            side 1   ; X = TCK count - 1
    out y, 1            side 1   ; Y = TMS level
    jmp !y tms_low      side 1
    set pins, 1         side 1   ; TMS high
    jmp bits            side 1
tms_low:
    set pins, 0         side 1   ; TMS low
bits:
    out null, 32        side 1   ; empty OSR so `pull ifempty` fetches data
loop:
    pull ifempty block  side 0   ; next TDI byte, TCK low
    out pins, 1         side 0   ; shift 1 TDI bit (2 cycles low)
    in pins, 1          side 1   ; sample 1 TDO bit, TCK high
    jmp x--, loop       side 1   ; decrement X (2 cycles high total)
    push block          side 1   ; push partial/empty last word; TCK high
.wrap
```

Each loop iteration is four PIO cycles: `pull ifempty`, `out`, `in`, `jmp`. TCK is low for the first two and high for the last two, so the TCK period is 4 cycles and the divider formula uses 4. `pull ifempty block` only stalls when the TX FIFO is empty; because the OSR was emptied with `out null, 32`, the first iteration of every entry always pulls the first TDI byte (or stalls until the CPU supplies it).

The `push block` at the end is unconditional and always executes. For `n` bits:

- the ISR autopushes one word for each complete 8 bits;
- the final `push block` pushes the remaining `r = n % 8` bits (or an empty word when `r == 0`).

So the RX FIFO receives `n/8 + 1` words per entry. The CPU's RX cursor uses exactly that formula.

7.6 The streaming protocol

Schema 7 — PIO entry format

```
TX FIFO (CPU -> PIO), per entry:
  word 0:  bits 0..6 = TCK count - 1 (0..63, i.e. 1..64 cycles)
           bit 7     = TMS level for the whole entry (0 or 1)
  word 1..: ceil(n / 8) bytes of TDI, LSB first, one byte per word

RX FIFO (PIO -> CPU), per entry:
  word 0..floor(n/8)-1 : full 8-bit TDO bytes, right-shifted into bits 31..24
  word floor(n/8)      : last r = n % 8 bits in bits 31..(32-r)
                         (empty when r == 0)

Example n = 13, TMS = 1, TDI = 0xA5, 0x1C:
  TX:  [0x8C] [0xA5] [0x1C]
        ^^^^ bits 0..6 = 12 (count-1), bit 7 = 1
  RX:  [byte0] [5 bits]
```

The header encoding is the reason `entry_header()` exists:

```
static inline uint32_t entry_header(uint32_t info, uint32_t *bits) {
    uint32_t n = info & JTAG_SEQUENCE_TCK;   // 0..63
    if (n == 0) n = 64;
    *bits = n;
    return (n - 1) | ((info & JTAG_SEQUENCE_TMS) ? 0x80 : 0);
}
```

A count of 64 is encoded as `0` in the 6-bit field, which is why the command format defines `0` as 64 (`JTAG_SEQUENCE_TCK` is `0x3F`). The TMS bit is bit 7 of the header, so the header byte written to the FIFO is not the same as the DAP `info` byte; only the count and TMS bits are used.

7.7 `pio_jtag_sequences()`: the CPU-side streaming loop

The function walks the same request buffer with two cursors:

- **TX cursor** `tp`: sends one header, then `ceil(n/8)` TDI bytes;
- **RX cursor** `rp`: skips the same entry structure but only consumes the TDO words, and only writes captured bytes for entries with `JTAG_SEQUENCE_TDO`.

```
while (t_left || t_bytes || r_left || r_words) {
    if (!tx_full) {
        if (t_bytes) { *txf = *tp++; t_bytes--; }
        else if (t_left) {
            *txf = entry_header(*tp++, &bits);
            t_bytes = (bits + 7) / 8;
            t_left--;
        }
    }
    if (r_words == 0 && r_left) {
        info = *rp;
        entry_header(info, &bits);
        r_capture = (info & JTAG_SEQUENCE_TDO) != 0;
        r_words = bits / 8 + 1;
        r_rem = bits % 8;
        rp += 1 + (bits + 7) / 8;
        r_left--;
    }
    if (r_words && !rx_empty) {
        val = *rxf; r_words--;
        if (r_capture) {
            if (r_words) *out++ = val >> 24;
            else if (r_rem) *out++ = val >> (32 - r_rem);
        }
    }
}
return (consumed << 16) | produced;
```

This is a cleaned-up excerpt: the real function calls `pio_sm_is_tx_fifo_full()` and `pio_sm_is_rx_fifo_empty()` and fetches the FIFO pointers from `g_pio_jtag` before the loop. The TX and RX FIFOs are four words deep each. The loop never blocks on a FIFO; it only writes when there is space and reads when there is data. Because the PIO is much faster than the CPU's byte-at-a-time feeding, the CPU is the bottleneck at high TCK, not the state machine.

The return value is the same packed form used by `DAP_JTAG_Sequence`: `(request_bytes_consumed << 16) | response_bytes_written`.

7.8 Handing pins between PIO and SIO

Some DAP commands must drive TCK/TDI/TMS from the CPU even while the PIO is enabled (JTAG IR scans, DP/AP transfers, `DAP_SWJ_Pins`, `DAP_SWJ_Sequence`). The handoff is explicit:

- `PIN_TCK_TDI_TO_SIO()` (in `DAP_config.h`) calls `pio_jtag_pins_to_sio()` if the PIO currently owns the pins.
- `pio_jtag_pins_to_sio()` reads the current pin levels, preloads the SIO output latch with those levels, sets the directions, then switches the pin function to `GPIO_FUNC_SIO`. Doing the latch before the function switch avoids a glitch.
- `pio_jtag_pins_to_pio()` calls `pio_gpio_init()` on the three pins again before the next sequence. The PIO is stalled on `pull` with TCK high, which matches the level the bit-banging path leaves, so the switch is clean.

`PIN_TCK_TDI_TO_SIO()` is called once per command, not per bit — SWD would pay for it on every bit otherwise.

7.9 What the PIO does not accelerate

Only `DAP_JTAG_Sequence` takes the PIO path. `JTAG_IR()`, `JTAG_Transfer()`, `JTAG_ReadIDCode()`, `JTAG_WriteAbort()`, `DAP_SWJ_Sequence` and all SWD remain SIO bit-banging. That is a deliberate scope decision:

- OpenOCD's JTAG traffic is dominated by `DAP_JTAG_Sequence` (up to hundreds of entries per 512-byte packet), so that is where the win is.

- `DAP_Transfer` is a strict DP/AP handshake with WAIT/FAULT turnaround; the bit-bang code already handles it, and the volume is lower.
- SWD has a bidirectional turnaround that the PIO program does not model.

The practical consequence is that `adapter speed` affects two clocks: the PIO divider for sequences and the SIO delay loop for everything else. They do not necessarily produce the same TCK at a given setting: the PIO has a defined upper bound of 31.25 MHz (`div = 1`), while the SIO paths are limited by the CPU loop and its delay calibration as the requested delay shrinks.

---

8. End-to-end: one `DAP_JTAG_Sequence` request

This section follows one 512-byte request from the host to the target and back. Assume OpenOCD sends a single `DAP_JTAG_Sequence` with several entries, at 10 MHz, while the DAP thread is spinning.

Schema 8 — timeline of one request

```
time   core 0 ("TUD")                          core 1 ("DAP")
────   ──────────────────────────────────────  ──────────────────────────────
t0     USB OUT packet 1 (64 B) arrives
       USB IRQ -> tud_event_hook_cb -> post usb_sem
t1     tud_task() drains the event
       dap_edpt_xfer_cb(OUT): out_len=64
       dap_req_len() says "more" -> arm OUT
t2..   packets 2..8 arrive; each callback
       re-arms the endpoint
t8     last packet completes (short or 512 B)
       __dmb(); USBRequestBuffer.wptr++
       post dap_sem
       arm OUT for the next request
t9                                             dap_thread() sees wptr != rptr
                                               __dmb(); rptr++
                                               (OUT stalled? no)
                                               DAP_ExecuteCommand()
                                                 -> DAP_JTAG_Sequence
                                                    -> pio_jtag_sequences()
                                                       feed headers/TDI
                                                       drain TDO words
t10                                            response ready
                                               wait for free response slot
                                               memcpy to USBResponseBuffer
                                               resp_len = N
                                               __dmb(); wptr++
                                               dap_kick_usb()
t11    (if mutex busy) usb_sem posted         (if mutex free) dap_edpt_service()
       TUD: dap_edpt_service()                  -> dap_edpt_start_in()
       -> dap_edpt_start_in()                   usbd_edpt_xfer(IN, N)
t12    IN transfer: up to 8 USB packets
       if N % 64 == 0 && N < 512 -> ZLP
t13    dap_edpt_xfer_cb(IN): free slot
       rptr++; post dap_sem; start next
```

The per-stage counters in `dap_stats` measure exactly these intervals:

- `queue_us` = t9 − t8 (request waiting for the DAP thread),
- `exec_us` = t10 − t9 (the actual JTAG work),
- `slot_wait_us` = wait for a free response slot,
- `inq_us` = t11 − t10 (response waiting for the IN endpoint),
- `in_us` = t12 − t11 (USB transfer),
- `gap_us` / `gap_idle_n` = OUT-to-OUT spacing.

For a real 512-byte `DAP_JTAG_Sequence` at 10 MHz the measured `exec_us` is about **292 µs**; the whole packet cycle is about **1,450 µs**, so most of the time is the USB round trip and the PC, not the probe (§ 10).

---

9. Build configuration

`CMakeLists.txt` is a Pico SDK project with one executable and one extra library target.

Item	Value
Board	`weact_studio_rp2040_16mb` (16 MB flash, LED GP25)
Binary type	`copy_to_ram`
Extra outputs	`.bin`, `.hex`, `.uf2`, `.dis`, `.map`
PIO header	generated from `src/jtag.pio` by `pico_generate_pio_header`
Includes	`CMSIS_6/CMSIS/Core/Include/`, `include/`, `src/`
Warnings	`-Wall`
Defines	`PICO_RP2040_USB_DEVICE_ENUMERATION_FIX=1`, `OS_USE_OS_APP_CONFIG_H`, `PICO_USE_MALLOC_MUTEX=0`
Libraries	`micro-os-plus::rp2040`, `pico_stdlib`, `pico_unique_id`, `tinyusb_device`, `tinyusb_board`, `hardware_pio`

`CMSIS_6/` is an uninitialised git submodule in this checkout; the include path is declared, but the CMSIS Core headers used by the build come from the Pico SDK's `cmsis_core` target (also linked by the µOS++ port).

The µOS++ port (`micro-os-plus/micro-os-plus-iii-rp2040/CMakeLists.txt`) is an INTERFACE library on top of the kernel:

- compiles `micro-os-plus/micro-os-plus-iii-rp2040/src/os-core-rp2040.cpp` (PendSV, per-core SysTick, kernel lock, IPI, core-1 launch);
- defines `OS_NCPU=2` and `OS_USE_SMP_SCHEDULER=1`;
- adds `-fno-use-cxa-atexit` for C++;
- adds linker symbols `_Heap_Begin=__end__` and `_Heap_Limit=__HeapLimit` so the kernel's `_sbrk()` uses the SDK's heap;
- links `cmsis_core`, `hardware_sync`, `hardware_timer`, `hardware_clocks`, `pico_multicore`.

`TINYUSB_OPT_OS OPT_OS_NONE` is set **before** `pico_sdk_init()`, which is what makes tinyusb compile without an OSAL. `OS_USE_OS_APP_CONFIG_H` makes the kernel read `src/cmsis-plus/os-app-config.h`:

Setting	Value
`OS_INTEGER_SYSTICK_FREQUENCY_HZ`	1000
`OS_INTEGER_RTOS_MAIN_STACK_SIZE_BYTES`	2048
`OS_INTEGER_RTOS_IDLE_STACK_SIZE_BYTES`	1024
`OS_INTEGER_RTOS_DEFAULT_STACK_SIZE_BYTES`	2048

The build outputs are:

File	Size	Use
`.elf`	1,803,708 B	Debugging, symbolication
`.bin`	60,108 B	Raw flash image
`.uf2`	120,320 B	Drag-and-drop to the RP2040 BOOTSEL drive

File	Size	Use
`.hex`	169,121 B	Alternative flash format
`.dis`	1,158,877 B	Full disassembly
`.elf.map`	—	Linker map

The `.bin` size equals the flash span `0x10000000..0x1000EACC`, which confirms that `.boot2` + `.flashtext` + `.binary_info` + `.text` (LMA) + `.data` (LMA) are contiguous.

The build is reproducible with:

```
export PICO_SDK_PATH=/path/to/pico-sdk
cmake -S . -B build -G Ninja -DPICO_BOARD=weact_studio_rp2040_16mb
cmake --build build
```

`PICO_BOARD` already defaults to the WeAct board in `CMakeLists.txt`; the command-line option is only needed to override it.

10. Performance model and measurements

The probe deliberately keeps the packet path outside the kernel. The cost model is simple:

```
packet cycle = USB OUT transfer
             + request queueing
             + DAP execution (JTAG/SWD work)
             + response queueing
             + USB IN transfer
             + host turnaround
```

The project's recorded measurements (Raspberry Pi 4 B host, `smp_mem_test`, `SEMI_PRINTS=1`, 512-byte packets; from `README.md` and `PERFORMANCE_ANALYSIS_AND_OPTIMIZATION.md`, not re-measured here) are:

Probe firmware	Adapter speed	`load_image` (74,888 B)	JTAG work/packet	Whole test
CPU bit-banging, 512 B	4000	1,528 ms (49.0 KB/s)	581 µs	308.1 s
PIO, TMS via SIO, wait per entry	4000	1,702 ms	755 µs	324.5 s
PIO, TMS via SIO, wait per entry	10000	1,578 ms	621 µs	315.3 s
PIO, TMS in stream (current)	**10000**	**1,286 ms (58.2 KB/s)**	**292 µs**	**291.5 s**
PIO, TMS in stream (current)	15000	1,287 ms (58.2 KB/s)	291 µs	285.1 s

Interpretation:

- **Streaming the entries halves the JTAG work** (581 → 292 µs). Waiting for the CPU to set TMS and restart the PIO after every entry is slower than bit-banging, even at a higher TCK.
- **15 MHz loads no faster than 10 MHz.** The PIO is no longer the limit; the CPU's byte-at-a-time FIFO feeding and TDO sorting, or the USB round trip, dominates.
- **Most of the cycle is not the probe.** At 10 MHz the JTAG work is 292 µs of a ~1,450 µs cycle; the rest is receiving the request in 64-byte USB packets and the PC's turnaround. With semihosting prints enabled, OpenOCD reading each string byte by byte still dominates the whole test (~0.9 s per line).

- **512-byte DAP packets are the other big win.** They cut the number of USB round trips by 8× and made `load_image` 23 % faster than the 64-byte build (about 47× the stock debugprobe).

The `dap_stats` counters are the tool used to attribute time. Read them with `cmsis-dap cmd 0x80 <sel>` and reset with `0x80 0xFF`. A useful sanity check after a load: `packets`, `exec_us / packets`, `in_us / in_n`, and `queue_us / packets`.

11. Inspecting and debugging the ELF

These commands reproduce every table in this document:

```
ELF=build/JTAGprobe_smp_pio_std_weact.elf
arm-none-eabi-readelf -h  $ELF        # header, entry point, flags
arm-none-eabi-readelf -l  $ELF        # program headers / load regions
arm-none-eabi-objdump -h  $ELF        # section VMA/LMA/size
arm-none-eabi-objdump -d -j .flashtext $ELF   # boot/copy code
arm-none-eabi-objdump -d -j .text $ELF        # all firmware code
arm-none-eabi-nm -S -n    $ELF | less # symbols with sizes, sorted
arm-none-eabi-size -A     $ELF        # section sizes
grep -n "SYMBOL" build/JTAGprobe_smp_pio_std_weact.elf.map
```

Useful things to look for:

- **Entry point** `0x100001e9` and the `data_cpy_table` at `0x10000238` prove the copy-to-RAM scheme.
- **Vector table**: `.flashtext` entries 14/15 point at `0x20005031` / `0x20004FE5`, i.e. the kernel's PendSV/SysTick handlers, not the SDK's weak ones.
- **Sizes**: `.bss` is dominated by `_vendord_itf` (16 KiB), `_cdcd_itf` (4.2 KiB), the two 2 KiB rings and the thread stacks. If RAM becomes tight, these are the first candidates.
- **`main` is weak** (`W main` in `nm`): the kernel's `main()` is the one linked, and the application entry is `os_main`.

On-target inspection with OpenOCD:

```
# Read the packet-timing counters (field 0 = packets)
openocd -f interface/cmsis-dap.cfg -f target/rp2040.cfg \
        -c "adapter speed 10000" \
        -c "init" -c "cmsis-dap cmd 0x80 0x00" -c "shutdown"
```

Common failure modes and where to look:

Symptom	First place to check
No USB device	`usb_descriptors.c` VID/PID, `tusb_init()`, `OPT_OS_NONE`, 12 MHz crystal/clock
Device enumerates but no DAP	Interface 0 class/subclass/protocol, `usbd_app_driver_get_cb`, endpoint addresses
DAP commands time out	`dap_thread` running on core 1, `dap_sem` posts, `usb_mutex` deadlock
JTAG works slowly	PIO claimed? `pio_jtag_is_enabled()`, divider, `DAP_SWJ_Clock`
JTAG sequences corrupt	TMS header bit, TDI byte count, RX word count, `pio_jtag_pins_to_pio()`
SWD fails	`PORT_SWD_SETUP()` ran, SWDIO direction, turnaround, nRESET
UART drops characters	UART thread priority, `interval`, FIFO depth, flow control
LED never blinks	

Symptom	First place to check
	`led_tck_activity()` called by DAP commands, timer IRQ alive

The LED has a deliberate 100 ms hold (`LED_TCK_HOLD_US`): a single command still produces a visible blink, and the timer toggles every 50 ms while TCK/SWCLK is in use. It is on solid when the probe is idle.

12. Source map and glossary

12.1 Source map

Path	Role
`src/main.c`	`os_main`, thread creation, USB thread, HID/CDC callbacks
`src/probe_os.h`	Shared µOS++ objects and the core/thread contract
`src/cmsis-plus/os-app-config.h`	Kernel tick and stack sizes
`src/tusb_edpt_handler.c/.h`	DAP class driver, rings, packet assembly, stats
`src/usb_descriptors.c`	Device/config/string/BOS/MS OS 2.0 descriptors
`src/tusb_config.h`	tinyusb buffer sizes and options
`src/DAP.c`	CMSIS-DAP command decoder and transfer algorithms
`src/DAP_vendor.c`	Vendor command `0x80` timing readout
`include/DAP.h`	DAP IDs, request bits, state, delay macros
`src/JTAG_DP.c`	JTAG IR/transfer/IDCODE/WriteABORT bit-banging, sequence fallback
`src/SW_DP.c`	SWD transfer, SWJ sequence, SWD sequence
`src/SWO.c`	SWO command code (compiled out with SWO disabled)
`src/pio_jtag.c` , `include/pio_jtag.h`	PIO claim/config/handoff/streaming
`src/jtag.pio`	The `jtag_tdo` state machine
`src/cdc_uart.c/.h`	USB CDC ↔ `uart1` bridge
`src/led.c/.h`	GP25 status LED and activity stamp
`src/get_serial.c/.h`	Flash unique ID → USB serial string
`include/DAP_config.h`	Board pins, port setup, pin macros, delays
`include/board_pico_config.h`	LED/UART pin and product-string configuration
`micro-os-plus/micro-os-plus-iii/`	µOS++ III SMP kernel (unmodified)
`micro-os-plus/micro-os-plus-iii-rp2040/`	RP2040 port: PendSV, SysTick, IPI, core-1 launch
`CMakeLists.txt`	Pico SDK build, µOS++ link, PIO header generation

12.2 Glossary

Term	Meaning
DAP	

Term	Meaning
	CMSIS-DAP, the USB debug protocol spoken by OpenOCD/pyOCD/probe-rs
DP / AP	ARM Debug Port / Access Port (the two-level debug interconnect)
DPACC / APACC	JTAG IR codes used to access the DP or AP
RDBUFF	DP read buffer; AP reads are posted through it
SPSC	Single-producer, single-consumer ring; no lock needed
PIO	RP2040 programmable I/O state machine
SIO	RP2040 single-cycle GPIO block (`gpio_set_mask`, `gpio_get_all`, …)
SM	PIO state machine
ZLP	USB zero-length packet, used to terminate a multiple-of-64 transfer
PendSV	ARM exception used for context switching
IPI	Inter-processor interrupt; here the SIO FIFO
XIP	Execute in place from flash (the opposite of `copy_to_ram`)
MSP / PSP	Main / process stack pointer; interrupts use MSP, threads use PSP
SCRATCH_X / Y	RP2040's two 4 KiB SRAM banks used by the SDK for core-1/core-0 stacks

Appendix A: request-length grammar

`dap_req_len()` is a hand-written parser for the subset of CMSIS-DAP that OpenOCD emits. In a compact grammar:

```
request        := command
command        := fixed | transfer | transfer_block | swj_sequence
                | swd_sequence | jtag_sequence | jtag_configure
                | queue | execute | unknown

fixed          := CMD_1 | CMD_2 | CMD_3 | CMD_5 | CMD_6 | CMD_7
CMD_1          := 0x03 | 0x07 | 0x0A | 0x1B
CMD_2          := 0x00 | 0x02 | 0x13 | 0x16 | 0x17 | 0x18 | 0x1A | 0x1E | 0x80
CMD_3          := 0x01 | 0x09 | 0x1C
CMD_5          := 0x11 | 0x19
CMD_6          := 0x04 | 0x08
CMD_7          := 0x10

transfer       := 0x05 index count { req [4-byte value] }
                  ; 4 bytes present unless RnW or MATCH_VALUE
transfer_block := 0x06 index count_lo count_hi req [4-byte value * count]
swj_sequence   := 0x12 bits { data }
                  ; bits == 0 means 256; data length = ceil(bits/8)
swd_sequence   := 0x1D count { info [data] }
jtag_sequence  := 0x14 count { info data }
jtag_configure := 0x15 count { ir_length }
queue          := 0x7E count { command }
execute        := 0x7F count { command }
unknown        := 1 byte
```

The actual function returns `0xFFFFFFFF` ("need more") until enough bytes have arrived; this is what lets the OUT endpoint decide whether to re-arm for another 64-byte USB packet. The recursion for Queue/Execute is the only non-trivial case: each sub-command is parsed with the same function, and the lengths are added until the count is satisfied.

Note that this grammar describes `dap_req_len()`, not the dispatch: `0x07` (`DAP_TransferAbort`) has a known length of 1, but `DAP_ProcessCommand()` has no handler for it (§6.2).

Appendix B: constants and addresses

B.1 CMSIS-DAP configuration

Constant	Value	Location
`DAP_PACKET_SIZE`	512 (v2)	`include/DAP_config.h`
`DAP_PACKET_COUNT`	4	`include/DAP_config.h`
`DAP_DEFAULT_SWJ_CLOCK`	1,000,000 Hz	`include/DAP_config.h`
`DAP_DEFAULT_PORT`	1 (SWD)	`include/DAP_config.h`
`DAP_JTAG_DEV_CNT`	8	`include/DAP_config.h`
`CPU_CLOCK`	125,000,000 Hz	`include/DAP_config.h`
`IO_PORT_WRITE_CYCLES`	1	`include/DAP_config.h`
`TIMESTAMP_CLOCK`	1,000,000 Hz	`include/DAP_config.h`
`SWO_UART`, `SWO_MANCHESTER`	0, 0	`include/DAP_config.h`
`DAP_FW_VER`	`"2.0.0"`	`include/DAP.h`

B.2 Runtime constants

Constant	Value	Meaning
`DAP_SPIN_US`	2000	DAP thread spin before blocking
`USB_SPIN_US`	2000	USB thread spin after last packet
`DAP_GAP_IDLE_US`	10000	OUT gap classified as host idle
`LED_BLINK_MS`	50	LED toggle period while active
`LED_TCK_HOLD_US`	100000	Activity hold time
`DAP_EP_SIZE`	64	Full-speed bulk packet
`THREAD_STACK_SIZE_BYTES`	2048	UART/TUD/DAP stacks
`OS_NCPU`	2	SMP CPU count
SysTick	1000 Hz	RTOS tick

B.3 Pin constants

Macro	GPIO	Note
`PROBE_PIN_nTRST`	0	fixed high after `DAP_SETUP`
`PROBE_PIN_nRESET`	1	open-drain
`PROBE_PIN_SWCLK_TCK`	2	PIO sideset / SIO
`PROBE_PIN_SWDIO_TMS`	3	PIO set / SIO

Macro	GPIO	Note
`PROBE_UART_TX`	4	`uart1`
`PROBE_UART_RX`	5	`uart1`
`PROBE_PIN_TDI`	6	PIO out / SIO
`PROBE_PIN_TDO`	7	PIO in / SIO input
`PROBE_USB_CONNECTED_LED`	25	status LED

B.4 Addresses

Symbol	Address
`.boot2`	`0x10000000`
`_entry_point`	`0x100001E8`
`_reset_handler`	`0x100001F6`
`.text` (runtime)	`0x200000C0`
`.data` (runtime)	`0x2000CDF0`
`.bss`	`0x2000E8E0`
`__end__` / heap start	`0x20017D64`
`__HeapLimit`	`0x20040000`
`__StackOneTop` (core 1)	`0x20041000`
`__StackTop` (core 0/MSP)	`0x20042000`
`DAP_ExecuteCommand`	`0x20001884`
`dap_thread`	`0x20003A44`
`usb_thread`	`0x200031BC`
`cdc_thread`	`0x20003F44`
`pio_jtag_sequences`	`0x200045A0`
`PendSV_Handler`	`0x20005030`
`SysTick_Handler`	`0x20004FE4`
`main`	`0x20006568`

End of document. For board-specific build instructions and Pico-vs-WeAct differences, see `WEACT_CONFIGURATION_BUILD_AND_DIFFERENCES.md`; for the higher-level architecture and measured optimization history, see `ARCHITECTURE_AND_IMPLEMENTATION.md`, `PERFORMANCE_ANALYSIS_AND_OPTIMIZATION.md` and `README.md`.

This PDF was generated from the Markdown source with:

```
pandoc ELF_ANATOMY_AND_RUNTIME_WALKTHROUGH.md \
    -o ELF_ANATOMY_AND_RUNTIME_WALKTHROUGH.pdf \
    --pdf-engine=weasyprint --css=elf_walkthrough_style.css --standalone \
    --metadata pagetitle="ELF Anatomy and Runtime Walkthrough"
```